OpenReview.net(/) Notifications (/notifications) Activity (/activity) Tasks (/tasks) Amit Afl...

Back to **Author Console** (/group?id=TMLR/Authors#your-submissions)

Fast and Memory-Efficient Wavelet Convolutions via I/O-Aware Reformulation

PDF (/pdf?id=oH2KYb6lhR)

Amit Aflalo (/profile?id=~Amit_Aflalo1), Shahaf E. Finder (/profile?id=~Shahaf_E._Finder1), Roy Amoyal (/profile?id=~Roy_Amoyal , Eran Treister (/profile?id=~Eran_Treister1), Oren Freifeld (/profile?id=~Oren_Freifeld1)

12 Aug 2026 (modified: 16 Aug 2026) Under review for TMLR Everyone Revisions (/revisions?id=oH2KYb6lhR) BibTeX Edit

CC BY 4.0 (https://creativecommons.org/licenses/by/4.0/)

Abstract: Wavelet convolution (WTConv) has emerged as an increasingly popular drop-in replacement for standard convolutions, expanding a network's receptive field exponentially with the number decomposition levels while keeping the parameter count linear. However, its reference implementation is severely memory-bound due to excessive data movement through high-bandwidth memory (HBM). We develop an I/O model of WTConv to characterize this bottleneck and use it to guide three algebraic reformulations: (1) recomputing the inexpensive Haar analysis butterfly on chip, (2) collapsing the multi-level synthesis cascade into a single closed-form pass indexed by output-coordinate bits, and (3) folding learned per-channel scales into the convolution weights. Together, these reformulations enable an I/O-aware fused implementation that substantially reduces HBM traffic. We evaluate the WTConvNeXt configuration across decomposition levels and a broad range of tensor shapes. Despite performing comparable arithmetic, the reference WTConv is substantially slower than the depthwise convolution it replaces. Our reformulation reduces modeled HBM traffic by approximately $2.55\times$, yielding up to a $4.35\times$ training speedup over the reference while roughly halving peak memory usage. Thus, our reformulation preserves the benefits of WTConv while substantially reducing its execution time and memory footprint, removing the systems overhead that previously limited its practical efficiency. Source code is available in the supplementary material.

Submission Type: Regular submission (no more than 12 pages of main content)
Supplementary Material: zip (/attachment?id=oH2KYb6lhR&name=supplementary_material)
Competing Interests: N/A
Human Subjects Reporting: N/A
Assigned Action Editor: Alexander Andreopoulos (/profile?id=~Alexander_Andreopoulos1)
Submission Number: 11285

Filter by reply type... Filter by author... Search keywords... Sort: Newest First − = ≡

Everyone | Editors In Chief | Paper11285 Action... | Paper11285 Authors | ✖ *4 / 4 replies show*

Add: **Withdrawal** **Official Comme**

Review of Paper11285 by Reviewer gCwa

Review by Reviewer gCwa 04 Sep 2026 at 15:23 (modified: 04 Sep 2026 at 15:23) Everyone Revisions (/revisions?id=bnbra7MvS2)

Summary Of Contributions:
The paper identifies intermediate-tensor materialization and high-bandwidth-memory traffic as the main bottleneck in the reference implementation of WTConv. It derives an element-level I/O model for reference WTConv and uses it to motivate an I/O-aware implementation. It introduces algebraic reformulations which can preserve the WTConv computation up to floating-point evaluation order. Key contributions include:

- Experiments report layer-level latency, memory, and traffic, as well as end-to-end ConvNeXt-T results on two GPUs.
- The paper provides a clear systems diagnosis backed by an explicit I/O model.
- It offers a useful algebraic derivation for the Haar synthesis cascade.
- It includes numerical checks for forward outputs and parameter gradients.
- It presents cumulative ablations and cross-hardware measurements.

Are the claims made in the submission supported by accurate, convincing and clear evidence?: Yes
Explain your answer above:
- The main claim that reference WTConv is memory-bound is supported by: the analytical I/O model, roofline analysis, hardware-counter measurements, and the observed dependence on library kernel selection.
- The claimed traffic reduction is supported by both the model and measured DRAM traffic.
- The latency and memory improvements are supported by broad sweeps over decomposition levels, tensor shapes, numerical precisions, kernel sizes, and two GPU architectures.
- The ablation study gives useful evidence that all three reformulations contribute to the final result.
- The numerical-agreement experiment supports equivalence up to floating-point reassociation error.

Would at least some individuals in TMLR's audience be interested in knowing the findings of this paper?: Yes
Explain your answer above:
- The I/O analysis and algebraic reformulation strategy is useful broadly.
- The paper is relevant to researchers using WTConv or other multi-stage, memory-bound operators.
- The bit-indexed synthesis derivation is technically interesting and may motivate related implementations for structured transforms.
- However, the audience is specialized because modern models does not widely use WTConv.

Requested Changes:
- Improve reproducibility by reporting exact GPU, driver, CUDA, compiler, PyTorch, and cuDNN versions; providing complete build commands and kernel/compiler settings; releasing executable code for main-table experiments; and documenting supplementary code structure and expected runtime. Strengthen baseline fairness by specifying the exact reference WTConv implementation, reporting whether torch.compile, vendor fusion, or other framework optimizations were tested, and explaining the rationale for each baseline's implementation and tuning choices.
- Report statistical variation in latency measurements by including standard deviation, confidence intervals, or per-run variation, and state the number of independent repeated runs used for the reported geometric means.
- Clarify whether reported FP16 differences affect loss or gradient scaling in realistic training, and add a brief end-to-end training stability comparison (e.g., loss curves) for reference and fused implementations under identical settings.
- Make the end-to-end evaluation self-contained by specifying exact ConvNeXt-T and WTConvNeXt-T configurations, software stack, and measurement protocol, and clarifying whether reported throughput includes data loading, synchronization, and optimizer-related work.

Broader Impact Concerns:
I do not perceive any concerns on its ethical implications.

Add: **Official Comment**

Review of Paper11285 by Reviewer 4vqy

Review by Reviewer 4vqy 26 Aug 2026 at 19:57 (modified: 04 Sep 2026 at 15:23) Everyone Revisions (/revisions?id=1GyiRhV3VC)

Summary Of Contributions:
The tackles a major practical issue with Wavelet Convolutions: despite having great theoretical parameter scaling, they are super slow in practice because they are often times memory-bound. The authors build a neat I/O model showing the arithmetic intensity is abysmal (~1.63 FLOP/byte). To fix this, they introduce three clever exact algebraic reformulations to collapse the multi-level synthesis cascade into a single closed-form pass.

Solid root-cause analysis. Moving away from FLOP counting to memory traffic is exactly how we look at these bottlenecks in production systems. The hardware validation (Nsight Compute) aligns perfectly with the theorical IO model.

Are the claims made in the submission supported by accurate, convincing and clear evidence?: Yes
Explain your answer above:
The authors don't just rely on theoretical models - they back them up with actual hardware performance counters. Tables 11 and 12 showing the measured vs predicted HBM traffic are exactly what I look for in systems-level ML papers. The cross-device validation on both an A6000 and a Blackwell GPU also proves the reformulations aren't just overfit to one specific architecture's cache hierarchy.

Would at least some individuals in TMLR's audience be interested in knowing the findings of this paper?: Yes
Explain your answer above:
There's a massive push in the community right now for efficient architectures, and I/O-aware operator fusion is a hot topic right now. Taking WTConv and removing the system overhead that stops it from being used in the real world is highly valuable to both ML researchers and infra folks.

Requested Changes:
The end-to-end inference throughput in Table 7 is still only 0.71x of ConvNeXt-T, even though the training throughput flipped to 1.06x. Could you expand a bit more on why inference doesn't see the same relative end-to-end win as training?

Broader Impact Concerns:
The paper focuses on computational efficiency, which generally has a positive impact by reducing energy consumption and GPU hrs.

Add: **Official Comment**

Review of Paper11285 by Reviewer JKBq

Review by Reviewer JKBq 21 Aug 2026 at 18:46 (modified: 04 Sep 2026 at 15:23) Everyone Revisions (/revisions?id=iQvD3CyWYA)

Summary Of Contributions:
This paper investigates an implementation to reduce memory footprint and increasing throughout for WTConv, a wavelet based convolution layer. The premise is that a vanilla implementation is very slow because it has excessive data movement on and off chip due to the many CUda calls, and proposes a totally on chip implementation instead. Numerics suggest about 3-4 x improvement in latency and throughout layer wise, and closer to 2-3 improvement in throughout when tested end to end.

Are the claims made in the submission supported by accurate, convincing and clear evidence?: Yes
Explain your answer above:
As far as I can tell the simple mathematical calculations are solid, no concerns

Would at least some individuals in TMLR's audience be interested in knowing the findings of this paper?: No
Explain your answer above:
This paper is primarily an implementation paper, with some calculations on the io and such. As far as I can tell there are no theoretical contributions; even Alg 1 is referenced from a previous paper. It is an appropriate systems paper for some place like ASPLOS, but my feeling is it not appropriate for TMLR.

Requested Changes:
I think if there was some algorithmic novelty, like a new way of implementing the wavelet transform that was both performance-strong and computationally efficient, it would be a more appropriate contribution

Broader Impact Concerns:
None

Add: **Official Comment**

Review Approval of Paper11285 by Action Editor GXmb

Review Approval by Action Editor GXmb (Alexander Andreopoulos (/profile?id=~Alexander_Andreopoulos1)) 16 Aug 2026 at 00:41 (modified: 16 Aug 2026 at 00:41)
Editors In Chief, Action Editors, Authors Revisions (/revisions?id=SDBVbr6iZh)

Under Review: Appropriate for Review

Add: **Official Comment**

About OpenReview (/about) Contact (/contact) FAQ (https://docs.openreview.net/getting-started/frequently-asked-questions)
Hosting a Venue (/group?id=OpenReview.net/Support) Sponsors (/sponsors) Terms of Use (/legal/terms) / Privacy Policy (/legal/privacy)
All Venues (/venues) **Donate** (/donate) News (/group?id=OpenReview.net/News&referrer=

id=OpenReview.net/News&referrer=
[Homepage](/))

OpenReview (/about) is a long-term project to advance science through improved peer review with legal nonprofit status. We gratefully acknowledge the support of the OpenReview Sponsors (/sponsors). © 2026 OpenReview